

Sprint 2 Development Report

ICBT Carpooling System — SEN5002 Agile Development and DevOps Project Report

1. Development Overview

Sprint 2 extends the foundational authentication and profile management architecture established in Sprint 1 by delivering core ride-discovery and interaction capabilities for the ICBT Carpooling System. The primary objective of Sprint 2 is to enable authenticated users to discover active ride offers matching their commute criteria and submit join requests to share rides. Building upon the Sprint 1 Node.js/Express backend, MongoDB native driver data access layer, JWT middleware, and React/Vite frontend UI design system, Sprint 2 introduces comprehensive search query filtering, privacy-conscious public offer inspection, robust join-request creation, and requester request-status tracking. In accordance with system design principles, access control operates strictly on token authentication, entity ownership (userId and requesterUserId), and action context—avoiding hardcoded role restrictions such as Driver or Passenger accounts. Consequently, any authenticated ICBT user can publish ride offers as an offer owner and request to join another user's offer as a requester.

2. Implemented Development Items

Table 4.2.1 details the functional features and technical components developed during Sprint 2, along with their respective sprint outcomes.

Table 4.2.1: Sprint 2 Implemented Development Items

Implemented Development Item	Sprint 2 Outcome
Searchable Active Ride-Offer Listing (US-13)	Developed GET /api/ride-offers/search API endpoint supporting multi-criteria filtering by origin, destination, date, and time window. Integrated case-insensitive MongoDB \$regex pattern matching and enforced filters to return only active offers with available seats (availableSeats > 0). Integrated custom PickerField dropdown components in the frontend search bar.
Public Offer Detail Viewing with Privacy Controls (US-14)	Implemented GET /api/ride-offers/public/:id endpoint returning complete route, schedule, seat availability, and status details. Applied a privacy serializer (toPublicRideOffer) to restrict owner metadata strictly to firstName and lastName, protecting sensitive contact details (email, phone, student ID).
Join-Request Creation & Persistence (US-15)	Created POST /api/join-requests API endpoint allowing authenticated users to submit a join request with an optional note. Implemented database insertion into the joinRequests collection with an initial status of Pending.
Business Rule Enforcement & Validation Guards (US-15)	Implemented validation logic in validators.js to enforce business constraints: (1) preventing duplicate pending requests for the same offer, (2) blocking self-requests where requesterUserId === ownerUserId, and (3) rejecting requests for inactive, full (0 seats), or cancelled offers.
Requester Request-Status Viewing (US-16)	Built GET /api/join-requests/mine API endpoint allowing users to view all join requests submitted under their account context (requesterUserId). Developed the MyRequestsPage.jsx dashboard displaying request history, embedded route summaries, and visual status badges (Pending, Accepted, Rejected).
Regression Stability & UI Consistency	Maintained complete backward compatibility with Sprint 1 auth, profile, and ride creation endpoints. Preserved the established UI component design tokens, custom pickers, and layout shells in styles.css.

3. API, Database, Security and Deployment Implementation

Table 4.2.2 summarizes the technical architecture, data storage structures, security guards, and deployment configurations implemented for Sprint 2.

Table 4.2.2: Sprint 2 Architecture, Security and Deployment Specifications

Area	Implementation Detail
Frontend Implementation	Developed using React and Vite. Updated SearchRidePage.jsx with custom PickerField selectors for Date and Time Window matching RideCreatePage.jsx styling; built OfferDetailPage.jsx modal for public offer inspection and request submission; created MyRequestsPage.jsx dashboard for requester status tracking.
Backend / API Implementation	Developed using Node.js and Express RESTful API routing pattern. Created endpoints: • GET /api/ride-offers/search • GET /api/ride-offers/public/:id • POST /api/join-requests • GET /api/join-requests/mine

Area	Implementation Detail
MongoDB Collections & Data Structures	Direct data access via MongoDB Node.js native driver (no ORM/ODM). Utilized collections: - rideOffers: Indexed on { status: 1, availableSeats: 1 } and \$regex text query fields. - joinRequests: Stores documents containing _id, rideOfferId, requesterUserId, ownerUserId, status ('Pending', 'Accepted', 'Rejected'), requestNote, requestedAt, and updatedAt. Indexed on { requesterUserId: 1 } and { rideOfferId: 1, requesterUserId: 1 }.
Authentication & Security Controls	Enforced requireAuth middleware on all Sprint 2 endpoints to verify JWT Bearer tokens. Enforced ownership checks on GET /api/join-requests/mine (requesterUserId === req.user._id). Applied privacy projection serializers (toPublicRideOffer) to strip email, phone number, student ID, and internal database keys before returning public offer data.
Validation & Business Rules	Implemented validateJoinRequest(payload, offer, existingRequests, requesterUserId) in validators.js enforcing: - Offer existence and active status (status === 'Active'). - Seat availability check (availableSeats > 0). - Self-request prevention (requesterUserId !== offer.userId). - Duplicate active request prevention (status !== 'Pending' for existing user request).
Deployment & Environment Configuration	Server containerized using Docker; backend deployed to Railway cloud hosting; frontend SPA built via Vite and deployed to Vercel hosting; CI pipeline automated via GitHub Actions executing linting and Jest test suites on push.

4. Developer Testing

Prior to handing off Sprint 2 functionality for QA review, automated developer unit and API integration tests were authored and executed using Jest and Supertest. Following Test-Driven Development (TDD) principles, tests were constructed to verify input normalization, boundary constraints, business validation rules, and security controls across all Sprint 2 components.

Table 4.2.3: Developer Automated Test Execution Matrix

Test Area	What Was Tested	Evidence Placeholder
Search Query Builder	Normalization of origin/destination strings (casing and whitespace handling) (UT-S2-01).	[Insert terminal output screenshot here]
Search Filtering	Verification that search results include only Active offers (UT-S2-02) and exclude zero-seat offers (UT-S2-03).	[Insert terminal output screenshot here]
Public Offer Detail Retrieval	Retrieval of route, date, time, and seat details (UT-S2-04) and verification that owner phone, email, and student ID are stripped (UT-S2-05).	[Insert terminal output screenshot here]
Join Request Validation	Persistence of valid Pending request in joinRequests collection (UT-S2-06).	[Insert terminal output screenshot here]
Duplicate Request Guard	Blocking of duplicate Pending join requests for the same offer by the same user (UT-S2-07).	[Insert terminal output screenshot here]
Self-Request Guard	Rejection of join requests where requesterUserId === ownerUserId (UT-S2-08).	[Insert terminal output screenshot here]
Unavailable Offer Guard	Rejection of requests submitted for full, inactive, or cancelled offers (UT-S2-09).	[Insert terminal output screenshot here]
Request Status Isolation	Verification that GET /api/join-requests/mine returns only requests matching the token holder's requesterUserId (UT-S2-10).	[Insert terminal output screenshot here]
Protected Endpoint Guard	Rejection of unauthenticated requests with HTTP 401 (UT-S2-11).	[Insert terminal output screenshot here]
Sprint 1 Regression Suite	Execution of all 24 Sprint 1 auth, profile, and ride tests to confirm zero regression (UT-S2-12).	[Insert terminal output screenshot here]

5. Development Evidence Summary

Table 4.2.4 lists the development evidence artifacts confirming source control history, automated test execution, continuous integration, runtime UI behavior, and security rule enforcement.

Table 4.2.4: Development Evidence Artifact Summary

Evidence Reference	What the Evidence Should Demonstrate
--------------------	--------------------------------------

Evidence Reference	What the Evidence Should Demonstrate
Figure S2-DEV-01	GitHub Commit History & Pull Request Evidence — Demonstrates feature branch development, commit messages, and clean pull request merge history for Sprint 2 features.
Figure S2-DEV-02	Local Automated Unit / API Test Execution Output — Demonstrates terminal screenshot of Jest execution confirming 71 passed tests across 5 test suites (sprint1-api.test.js, sprint2-api.test.js, sprint2-tdd-spec.test.js, auth.test.js, validators.test.js).
Figure S2-DEV-03	GitHub Actions CI Workflow Result — Demonstrates successful automated continuous integration pipeline execution, running lint checks and Jest unit tests on push.
Figure S2-DEV-04	Runtime Application UI Screens — Demonstrates application interface screenshots: (a) Find a Ride search form with custom pickers, (b) Public Offer Detail modal, (c) Join Request form, and (d) My Requests dashboard displaying request status badges.
Figure S2-DEV-05	Browser Developer Tools Network Evidence — Demonstrates Chrome DevTools Network tab captures confirming correct RESTful API endpoint calls and JSON payloads.
Figure S2-DEV-06	Security & Ownership Validation Log — Demonstrates server HTTP response logs verifying enforcement of HTTP 409 Conflict (duplicate request), 422 Unprocessable Entity (self-request / full offer), and 401 Unauthorized (missing token).

6. Known Limitations / Carry Forward

While Sprint 2 successfully delivered the ride-discovery, join-request creation, and requester status-tracking workflow, certain downstream features were intentionally scoped for subsequent iterations:

1. Offer-Owner Decision Workflow (Sprint 3 Scope): In Sprint 2, join requests remain in a Pending state. The capability for offer owners to review pending requests, approve or reject requesters, and automatically decrement availableSeats upon acceptance is scheduled for implementation in Sprint 3.
2. Real-Time Status Notifications: Request status changes currently refresh on navigation or manual dashboard reload. Automated web-socket or push notifications for status updates are carried forward to future enhancements.
3. Advanced Distance/Map Filtering: Search filters currently operate on route text criteria (origin and destination). Geospatial spatial proximity search is identified as a potential post-release enhancement.